Course Name:	Web Development Laboratory (316U01L306)	**Semester:**	**III**
Date of Performance:	**25/08/2026**	**DIV/ Batch No:**	**A2**
Student Name:	**Kehan Desai**	**Roll No:**	**16010125050**

Experiment No: 3

Title: To implement a Version Control System using Git and GitHub for managing, tracking, and sharing the website files developed in Experiments 1 and 2.

Aim of the Experiment: Implementation of Version Control System (Git & GitHub)

Objectives for the Experiment:

1. Understand the concept and importance of version control in web development.
2. Install and configure Git and create a local Git repository.
3. Track website files using the working directory, staging area, and local repository.
4. Use essential Git commands such as init, status, add, commit, log, diff, branch, remote, push, pull, and clone.
5. Create and manage a remote repository on GitHub.
6. Upload the Experiment 1 and Experiment 2 case-study files to GitHub using meaningful commits.
7. Demonstrate synchronization between a local repository and GitHub.

COs to be achieved:

CO1: Developing webpages using HTML and CSS

Books/ Journals/ Websites references:

1. Git Documentation: https://git-scm.com/docs
2. GitHub Documentation: https://docs.github.com/
3. Git Exercises: https://gitexercises.fracz.com/

Theory:

Version Control

Version control is a system used to track and manage changes made to files during the development of a project. It allows developers to maintain different versions of a project and return to an earlier version when required.

Git

Git is a distributed version control system used to track changes in source code and other project files. It maintains a history of changes through commits.

GitHub

GitHub is a cloud-based platform that hosts Git repositories. It allows developers to store projects online, collaborate with others, and manage different versions of a project.

Git Repository

A repository is a location where Git stores the files and the history of changes made to those files.

Basic Git Workflow

The basic Git workflow can be represented as:

Working Directory → Staging Area → Local Repository → Remote Repository (GitHub)

Important Git Commands

Students should explain the purpose and syntax of the following commands:

Command	Purpose
git init	Initializes a new Git repository
git status	Displays the current state of the repository
git add	Adds files to the staging area
git commit	Saves staged changes in the local repository
git log	Displays commit history

git diff	Displays changes made to files
git branch	Displays or manages branches
git remote	Manages connections to remote repositories
git push	Uploads local commits to GitHub
git pull	Retrieves and integrates changes from GitHub
git clone	Creates a local copy of a remote repository

Problem statement:

Upload and maintain the website files developed for Experiments 1 and 2 of the assigned case studies using Git and GitHub. Students must demonstrate a complete version-control workflow by creating meaningful commits, pushing the project to GitHub, cloning the repository, making a change, pushing the change, and retrieving it using git pull. All tasks are compulsory.

Code:

Task 1: Create and Configure a GitHub Account

1. Create/sign in to a GitHub account using the student's own name "Your-Name" and email address "your-email@example.com".
2. Create a repository with a meaningful name related to the assigned case study, for example: smart-agriculture_GrpNo.
3. Add a suitable repository description. Do not upload unrelated files.

Task 2: Install and Configure Git

1. Install Git and open Git Bash.
2. Verify installation: git --version
3. Configure identity: git config --global user.name "Your-Name"
4. Configure email: git config --global user.email "your-email@example.com"
5. Verify configuration: git config --list

Task 3: Prepare and Initialize the Local Project

1. Create/open the folder containing the Experiment 1 and Experiment 2 website files. If required, create a project folder: mkdir smart-agriculture_GrpNo

2. Enter the folder: cd smart-agriculture_GrpNo
3. Confirm the files are present, such as index.html, style.css, script.js, images, and other files required by the case study.
4. Initialize Git: git init
5. Check the repository: git status
6. Observe the files shown as untracked.

Task 4: Track Files and Create the Initial Commit

1. Stage the project files: git add .
2. Check staging status: git status
3. Create a meaningful first commit: git commit -m "Initial case study website"
4. View history: git log

Task 5: Modify the Website and Create a Second Commit

1. Modify the website according to the assigned case study, such as changing content, adding a section, updating navigation, or modifying CSS.
2. Check changes: git status
3. View differences: git diff
4. Stage the changes: git add .
5. Commit them: git commit -m "Updated homepage and styling"
6. View the history again: git log

Task 6: Connect the Local Repository to GitHub

1. Copy the URL of the GitHub repository.
2. Add the remote: git remote add origin <GitHub-Repository-URL>
3. Verify: git remote -v
4. Set the primary branch to main if required: git branch -M main

Task 7: Push the Project to GitHub

1. Upload the local commits: git push -u origin main
2. Open the GitHub repository and verify that the website files, project structure, and commit history are visible.
3. Record the GitHub repository URL in the observation/deliverable section.

Task 8: Clone and Synchronize the Repository

1. Clone the repository into another folder: git clone <GitHub-Repository-URL>
2. Enter the cloned folder and make a small, identifiable website change.
3. Stage and commit: git add . ; git commit -m "Updated website content"

4. Push the change: git push
5. Return to the original project folder and retrieve the latest changes: git pull
6. Verify that the change appears in the original local project.

Commands to be used for the above tasks:

```
git --version
 git config --global user.name "Your-Name"
 git config --global user.email "your-email@example.com"
 git config --list

 mkdir smart-agriculture_GrpNo
 cd smart-agriculture_GrpNo
 git init
 git status
 git add .
 git commit -m "Initial case study website"
 git log

 # After making changes
 git status
 git diff
 git add .
 git commit -m "Updated homepage and styling"
 git log

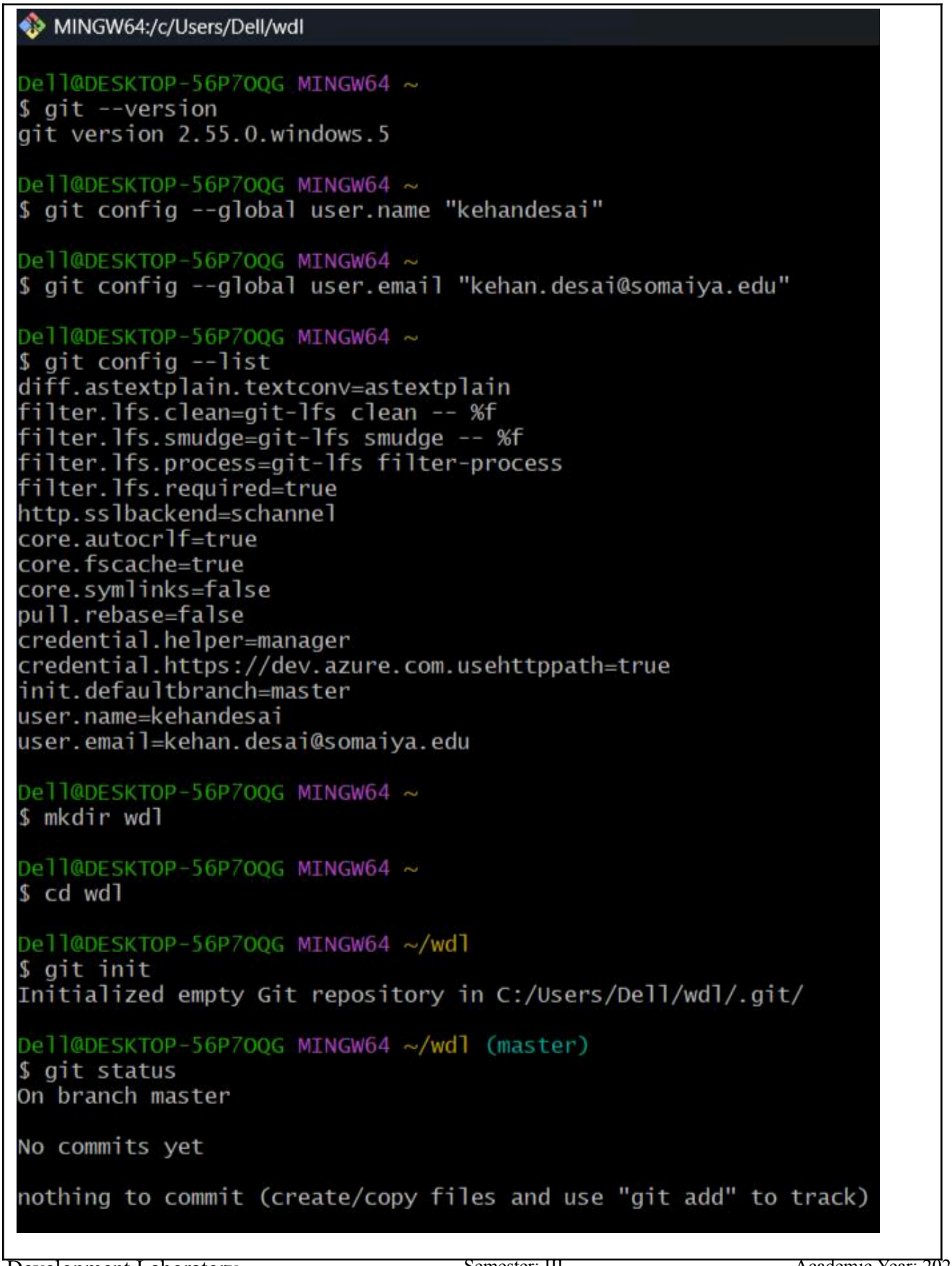
 # Connect to GitHub
 git remote add origin <GitHub-Repository-URL>
 git remote -v
 git branch -M main
 git push -u origin main

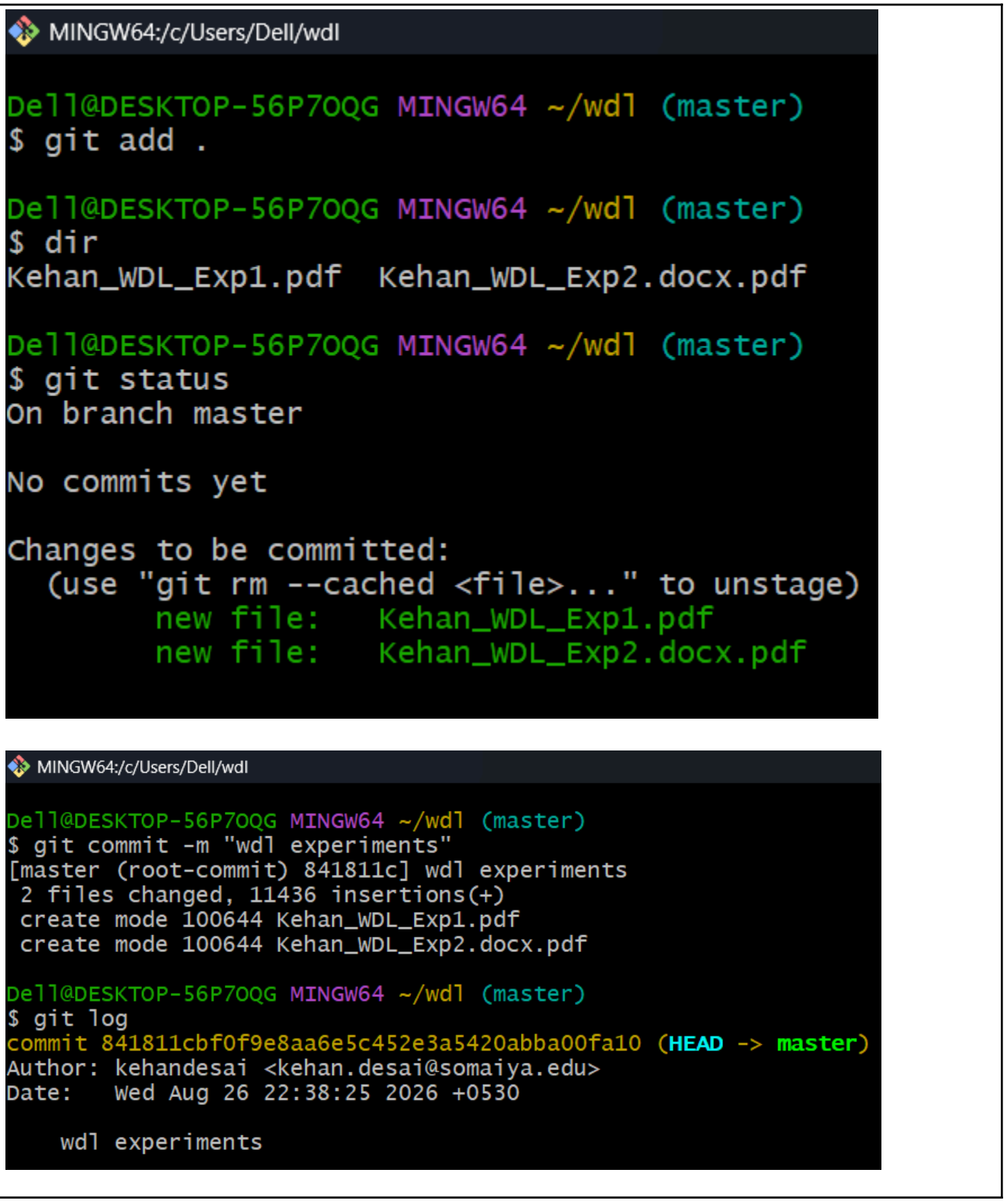
 # Clone and synchronize
 git clone <GitHub-Repository-URL>
 git add .
 git commit -m "Updated website content"
 git push
 git pull
```

Expected Output / Screenshots:(Paste here):

Steps are shown below for reference as per the above tasks.

- Git version displayed successfully.
- Git username and email configuration displayed.
- git status showing the project files.
- Successful initial commit and commit history using git log.
- Changes displayed using git diff.
- Remote URL displayed using git remote -v.
- Successful git push to GitHub.
- GitHub repository showing the website files and commit history.
- Successful git clone and git pull demonstrating synchronization.


MINGW64:/c/Users/Dell/wdl
Dell@DESKTOP-56P7OQG MINGW64 ~
$ git --version
git version 2.55.0.windows.5
Dell@DESKTOP-56P7OQG MINGW64 ~
$ git config --global user.name "kehandesai"
Dell@DESKTOP-56P7OQG MINGW64 ~
$ git config --global user.email "kehan.desai@somaiya.edu"
Dell@DESKTOP-56P7OQG MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=kehandesai
user.email=kehan.desai@somaiya.edu
Dell@DESKTOP-56P7OQG MINGW64 ~
$ mkdir wdl
Dell@DESKTOP-56P7OQG MINGW64 ~
$ cd wdl
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl
$ git init
Initialized empty Git repository in C:/Users/Dell/wdl/.git/
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git status
On branch master
No commits yet
nothing to commit (create/copy files and use "git add" to track)


MINGW64:/c/Users/Dell/wdl
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git add .
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ dir
Kehan_WDL_Exp1.pdf Kehan_WDL_Exp2.docx.pdf
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git status
On branch master
No commits yet
Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file: Kehan_WDL_Exp1.pdf
new file: Kehan_WDL_Exp2.docx.pdf
MINGW64:/c/Users/Dell/wdl
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git commit -m "wdl experiments"
[master (root-commit) 841811c] wdl experiments
2 files changed, 11436 insertions(+)
create mode 100644 Kehan_WDL_Exp1.pdf
create mode 100644 Kehan_WDL_Exp2.docx.pdf
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git log
commit 841811cbf0f9e8aa6e5c452e3a5420abba00fa10 (HEAD -> master)
Author: kehandesai <kehan.desai@somaiya.edu>
Date: Wed Aug 26 22:38:25 2026 +0530
wdl experiments

```
Dell@DESKTOP-56P7OQG MINGW64 ~
$ pwd
/c/Users/Dell

Dell@DESKTOP-56P7OQG MINGW64 ~
$ ls
'3D Objects'/
 AppData/
'Application Data'@
 Contacts/
 Cookies@
 Documents/
 Downloads/
 Favorites/
 IntelGraphicsProfiles/
 Links/
'Local Settings'@
 Music/
'My Documents'@
 NTUSER.DAT
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TM.blf
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000001.regtrans-ms
 NTUSER.DAT{2ad838bc-efea-11ee-a54d-000d3a94eaa1}.TMContainer00000000000000000002.regtrans-ms
 NetHood@
 OneDrive/
 PrintHood@
 Recent@
'Saved Games'/
 Searches/
 SendTo@
'Start Menu'@
 Templates@
 Videos/
 ntuser.dat.LOG1
 ntuser.dat.LOG2
 ntuser.ini
 update.txt
 wdl/

Dell@DESKTOP-56P7OQG MINGW64 ~
$ cd wdl

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ ls -a
./  ../  .git/  Kehan_WDL_Exp1.pdf  Kehan_WDL_Exp2.docx.pdf

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git status
On branch master
nothing to commit, working tree clean
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ echo "Experiment 3 update" > update.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        update.txt

nothing added to commit but untracked files present (use "git add" to track)

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git add .
warning: in the working copy of 'update.txt', LF will be replaced by CRLF the next time Git touches it

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   update.txt
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git commit -m "Updated project files"
[master d84762e] Updated project files
 1 file changed, 1 insertion(+)
 create mode 100644 update.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git log
commit d84762e261ba7ebb0dc7d42d820d52b87667fe90 (HEAD -> master)
Author: kehandesai <kehan.desai@somaiya.edu>
Date:   Wed Aug 26 22:52:00 2026 +0530

    Updated project files

commit 841811cbf0f9e8aa6e5c452e3a5420abba00fa10
Author: kehandesai <kehan.desai@somaiya.edu>
Date:   Wed Aug 26 22:38:25 2026 +0530

    wdl experiments
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git remote add origin https://github.com/kehandesai/wdl.git

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git remote -v
origin  https://github.com/kehandesai/wdl.git (fetch)
origin  https://github.com/kehandesai/wdl.git (push)

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (master)
$ git branch -M main
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ rm -rf wdl_clone

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git clone https://github.com/kehandesai/wdl.git wdl_clone
Cloning into 'wdl_clone'...
warning: You appear to have cloned an empty repository.

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ cd wdl_clone

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ echo "Testing clone repository" >> update.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        update.txt

nothing added to commit but untracked files present (use "git add" to track)

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ git add .
warning: in the working copy of 'update.txt', LF will be replaced by CRLF the next time Git touches it

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ git commit -m "Updated from clone"
[main (root-commit) f9fd4d1] Updated from clone
 1 file changed, 1 insertion(+)
 create mode 100644 update.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl/wdl_clone (main)
$ git push
```

GitHub Repository URL: https://github.com/kehandesai/wdl.git

Post Lab Subjective/Objective type Questions:

1. What is the purpose of git clone? How is it different from downloading a ZIP file from GitHub?

The git clone command is used to create a local copy of a remote Git repository available on platforms like GitHub. It downloads all project files along with complete version history, branches, commits, and remote repository information. This allows developers to track changes, create new branches, make updates, and synchronize their work with the original repository.
Downloading a ZIP file from GitHub only provides a snapshot of the project files at a particular point in time. It does not include commit history, branches, or Git tracking features. Therefore, git clone is more useful for development and collaboration, while a ZIP file is only suitable for viewing or using the current files.

2. Complete at least the relevant interactive exercises from Git Exercises (https://gitexercises.fracz.com/)and demonstrate basic branching, conflict resolution, and remote-repository operations.

```
MINGW64:/c/Users/Dell/wdl

Dell@DESKTOP-56P7OQG MINGW64 ~
$ cd /c/Users/Dell/wdl

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        wdl_clone/

nothing added to commit but untracked files present (use "git add" to track)

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git checkout -b feature
Switched to a new branch 'feature'

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (feature)
$ git branch
* feature
  main

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (feature)
$ echo "Feature Branch Content" > branch.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (feature)
$ git add .
warning: in the working copy of 'branch.txt', LF will be replaced by CRLF the next time Git touches it
warning: adding embedded git repository: wdl_clone
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> wdl_clone
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached wdl_clone
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (feature)
$ git commit -m "Added feature file"
[feature ae0cc1f] Added feature file
 2 files changed, 2 insertions(+)
 create mode 100644 branch.txt
 create mode 160000 wdl_clone

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (feature)
$ git checkout main
warning: unable to rmdir 'wdl_clone': Directory not empty
Switched to branch 'main'
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ echo "Main Branch Content" > branch.txt

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git add .
warning: in the working copy of 'branch.txt', LF will be replaced by CRLF the next time Git touches it
warning: adding embedded git repository: wdl_clone
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> wdl_clone
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached wdl_clone
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
gitgit
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git commit -m "Added main version"
[main 26b1390] Added main version
 2 files changed, 2 insertions(+)
 create mode 100644 branch.txt
 create mode 160000 wdl_clone

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git merge feature
Auto-merging branch.txt
CONFLICT (add/add): Merge conflict in branch.txt
Automatic merge failed; fix conflicts and then commit the result.

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main|MERGING)
$ branch.txt
bash: branch.txt: command not found

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main|MERGING)
$ git add .

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main|MERGING)
$ git commit -m "Resolved merge conflict"
[main 2ce1062] Resolved merge conflict

Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin main

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
```

```
Dell@DESKTOP-56P7OQG MINGW64 ~/wdl (main)
$ git pull
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=origin/<branch> main
```

Conclusion and Discussion:

Students should write the conclusion in their own words based on the work performed. The discussion should mention what was learned about Git/GitHub, how version history helped in tracking website changes, any difficulties encountered, and how those difficulties were resolved.

In this experiment, I learned the fundamentals of Git and GitHub, including repository management, cloning, branching, merging, and remote repository handling. Git provides an efficient way to manage source code by maintaining a history of changes and allowing developers to collaborate easily. Through practical exercises, I understood how version control helps in tracking modifications, restoring previous versions, and managing multiple contributors in a project.

Difficulties Faced:
Initially, understanding branch management and resolving merge conflicts was challenging. By practicing different Git commands and analyzing conflict messages, these difficulties were resolved. The experiment improved my understanding of how Git is used in real-world software development.

Peer feedback (minimum 3 classmates):
1. The experiment was well explained and helped in understanding the basics of Git commands, branching, and repository management.

2. The practical exercises were useful for learning how to handle version control, merge conflicts, and collaborate using GitHub.

3. The demonstration of Git operations was clear and improved understanding of real-world software development practices.